



KitaplıkApp

Kişisel Dijital Kütüphane — Proje Dokümantasyonu

Platform: Android (API 24+)

Dil: Kotlin

Veritabanı: Firebase Firestore

Auth: Firebase Authentication

API: Google Books API

Süre: 5 Hafta

1. Proje Genel Bakışı

KitaplıkApp, kullanıcıların kendi kişisel kitap koleksiyonlarını dijital ortamda yönetmelerine olanak tanıyan bir Android mobil uygulamasıdır. Uygulama; kitap ekleme, okuma durumu takibi, etiketleme, filtreleme, sıralama ve okuma istatistikleri gibi temel kütüphane yönetimi işlevlerini sunmaktadır.

1.1 Projenin Amacı

- Okunan, okunmakta olan ve okunmak istenen kitapları düzenli biçimde takip etmek
- Kitaplara kişisel etiket ve kategori ekleyerek koleksiyonlar oluşturmak
- Google Books API ile geniş bir kitap veritabanına erişim sağlamak
- Barkod okuyucu ile hızlı kitap ekleme imkânı sunmak
- Kullanıcıya okuma alışkanlıkları hakkında istatistiksel geri bildirim vermek

1.2 Hedef Kitle

- Kitap okumayı seven ve koleksiyonunu dijital ortamda takip etmek isteyen bireysel kullanıcılar
- Kişisel kütüphanesini yazar, tür veya seriye göre organize etmek isteyen okuyucular

1.3 Teknoloji Yığını

Katman	Teknoloji	Açıklama
Programlama Dili	Kotlin	Android geliştirme için modern, önerilen dil
UI Tasarımı	XML Layouts + Jetpack	Fragment, Navigation Component, ViewBinding
Mimari	MVVM	Model-View-ViewModel; kodun sürdürülebilirliği için
Kimlik Doğrulama	Firebase Authentication	E-posta/şifre tabanlı giriş, kayıt, şifre sıfırlama
Veritabanı	Firebase Firestore	Bulut tabanlı NoSQL gerçek zamanlı veritabanı
Kitap Arama	Google Books API	ISBN, isim ve yazar ile geniş kitap veritabanı
Barkod Okuma	ML Kit Barcode Scanning	Kamera ile ISBN barkodu okuma
Bağımlılık Yönetimi	Gradle + libs.versions	Modüler proje yapısı

2. Mimari ve Proje Yapısı

2.1 MVVM Mimarisi

Proje, Android geliştirme için Google tarafından önerilen MVVM (Model-View-ViewModel) mimarisi üzerine kurulacaktır. Bu mimari üç temel katmandan oluşur:

- View (UI Katmanı):** Kullanıcı arayüzü bileşenlerini barındırır. Activity ve Fragment sınıfları bu katmana aittir. Kullanıcı etkileşimlerini alır, ViewModel'den gelen verileri ekranda gösterir.
- ViewModel:** UI ile veri katmanı arasındaki köprüdür. LiveData veya StateFlow kullanarak UI'yi güncel tutar. İş mantığını (business logic) içerir.

- **Model (Veri Katmanı):** Veri kaynaklarını (Firestore, Google Books API) yönetir. Repository ve Model sınıflarını içerir. ViewModel ile doğrudan konuşmaz; yalnızca veri sağlar.

2.2 Klasör Yapısı

```
app/src/main/java/com/kitaplik/  
├── ui/  
│   ├── auth/           → Giriş, kayıt, şifre sıfırlama ekranları  
│   ├── library/        → Ana kütüphane ekranı, 3'lü sekme  
│   ├── search/         → Google Books arama ekranı  
│   ├── detail/         → Kitap detay sayfası  
│   ├── stats/          → İstatistik ekranı  
│   └── profile/        → Kullanıcı profili  
├── viewmodel/          → Her ekran için ViewModel sınıfları  
├── repository/         → FirestoreRepository, BooksApiRepository  
├── model/              → Book, User, Tag veri modelleri  
├── network/            → Google Books API Retrofit servisi  
└── util/               → Yardımcı fonksiyonlar, sabitler
```

2.3 Firebase Veri Modeli (Firestore)

Firestore'da hiyerarşik bir koleksiyon yapısı kullanılacaktır:

```
users/                                ← Koleksiyon  
  {userId}/                          ← Kullanıcı dokümanı  
    displayName: 'Ali'  
    email: 'ali@email.com'  
    readingGoal: 24                  ← Yıllık hedef (kitap sayısı)  
    createdAt: Timestamp  
  
  books/                             ← Alt koleksiyon  
    {bookId}/  
      title: 'Yüzüklerin Efendisi'  
      author: 'J.R.R. Tolkien'  
      isbn: '9780261103573'  
      coverUrl: 'https://...'  
      status: 'read'                ← 'reading' | 'to_read' | 'read'  
      rating: 5                     ← 0-5  
      note: 'Harika bir seri!'  
      tags: ['Tolkien', 'Fantastik', 'Klasik']  
      pageCount: 1178  
      addedAt: Timestamp  
      finishedAt: Timestamp
```

3. Kimlik Doğrulama Sistemi

Uygulama, Firebase Authentication kullanarak güvenli kullanıcı yönetimi sağlar. Tüm veriler kullanıcıya özeldir; başka kullanıcıların verilerine erişilemez.

3.1 Ekranlar ve İşlevler

Kayıt Ol Ekranı

Ad Soyad: Kullanıcının görünen adı — Firestore'a kaydedilir

E-posta: Geçerlilik kontrolü yapılır (@ ve domain zorunlu)

Şifre: Minimum 6 karakter kuralı Firebase tarafından uygulanır

Şifre tekrar: Eşleşme kontrolü istemci tarafında yapılır

Hata yönetimi: E-posta zaten kayıtlı, geçersiz format gibi durumlar gösterilir

Giriş Yap Ekranı

E-posta + Şifre: Firebase Auth ile doğrulama

Oturum kalıcılığı: Uygulama kapatılsa bile giriş hatırlanır

Hata yönetimi: Yanlış şifre, kayıtlı olmayan e-posta mesajları

Yönlendirme: Başarılı girişte doğrudan Ana Ekran'a geçiş

Şifremi Unuttum

Çalışma mantığı: Kullanıcı e-postasını girer

Firestore işlemi: sendPasswordResetEmail() ile sıfırlama bağlantısı gönderilir

Kullanıcı deneyimi: "E-postanızı kontrol edin" onay mesajı gösterilir

Güvenlik: Kayıtlı olmayan e-posta için de aynı mesaj gösterilir (bilgi sızıntısı önlenir)

3.2 Oturum Yönetimi

Firestore Auth, oturum durumunu otomatik olarak yönetir. Uygulamanın her açılışında FirebaseAuth.getInstance().currentUser kontrolü yapılır:

- Kullanıcı giriş yapmışsa → direkt Ana Ekran'a yönlendirme
- Kullanıcı giriş yapmamışsa → Giriş Ekranı'na yönlendirme
- Çıkış işlemi → FirebaseAuth.signOut() + Giriş Ekranı'na yönlendirme

4. Kütüphane Yönetimi

4.1 Üç Durumlu Okuma Takip Sistemi

Her kitap, kullanıcının okuma sürecindeki konumunu temsil eden üç durumdan birinde bulunur:

Okuyorum	Okumak İstiyorum	Okudum
Şu an okunmakta olan kitaplar	İleride okunacak kitaplar ("to-read list")	Tamamlanan kitaplar

Kullanıcı, bir kitabın durumunu detay sayfasındaki açılır menüden veya kütüphane listesindeki hızlı eylem butonlarından değiştirebilir. Durum değişikliği anında Firestore'a kaydedilir.

4.2 Kitap Ekleme Yöntemleri

4.2.1 Google Books API ile Arama

- Kullanıcı arama kutusuna kitap adı veya yazar adı yazar
- API isteği gönderilir: GET
https://www.googleapis.com/books/v1/volumes?q={sorgu}&key={API_KEY}
- Sonuçlar RecyclerView ile listelenir (kapak, başlık, yazar)
- Seçilen kitabın bilgileri otomatik doldurulur; kullanıcı durum ve etiket ekleyerek kaydeder

4.2.2 Barkod (ISBN) ile Ekleme

- Kullanıcı kamera simgesine dokunur
- ML Kit Barcode Scanning kütüphanesi kamera görüntüsünden ISBN numarasını okur
- ISBN, Google Books API'ye gönderilir: q=isbn:{ISBN}
- Kitap bilgileri otomatik olarak forma doldurulur
- Kullanıcı yalnızca durum ve etiket seçip kaydeder

4.2.3 Manuel Ekleme

- API sonucu bulunamadığında kullanıcı kitap bilgilerini elle girebilir
- Zorunlu alanlar: Başlık. İsteğe bağlı: Yazar, sayfa sayısı, yayın yılı

5. Etiket ve Kategori Sistemi

Etiket sistemi, kullanıcının kitaplarını kendi belirlediği kategorilere göre organize etmesini sağlar. Sabit kategoriler yerine tamamen serbest etiketleme tercih edilmiştir; bu sayede kullanıcı kendi koleksiyon mantığını oluşturabilir.

5.1 Nasıl Çalışır?

- Her kitaba birden fazla etiket eklenebilir (örnek: ['Tolkien', 'Fantastik', 'Klasik', 'Favori'])
- Etiketler Firestore'da kitap dokümanının içinde bir dizi (array) olarak saklanır
- Yeni etiket girildiğinde otomatik tamamlama, kullanıcının daha önce oluşturduğu etiketleri önerir
- Etiketler kütüphane ekranında chip (yuvarlak etiket) olarak görünür; dokunulduğunda filtre uygulanır

5.2 Koleksiyon Örneği

Tolkien Koleksiyonu — Örnek Senaryo

Kullanıcı 'Yüzüklerin Efendisi: Yüzük Kardeşliği' kitabına şu etiketleri ekler:

Tolkien, Fantastik, Klasik, Favori, Seri

Aynı etiketleri serinin diğer iki kitabına da ekler.

Kütüphane ekranında 'Tolkien' etiket chip'ine dokunur:

→ Yalnızca bu 3 kitap listelenir — koleksiyon otomatik oluşmuştur.

5.3 Etiket Yönetim Ekranı

- Kullanıcının oluşturduğu tüm etiketleri listeler
- Her etiketin yanında o etiketle işaretlenmiş kitap sayısı gösterilir
- Kullanılmayan etiketleri silme imkânı sunar
- Etiket adını düzenleme: Ad değiştiğinde tüm kitaplardaki etiket otomatik güncellenir (Firestore batch write)

6. Kütüphane Arama, Filtreleme ve Sıralama

6.1 Kütüphane İçi Arama

Bu özellik, kullanıcının Google Books'ta değil kendi kütüphanesindeki kitaplarda arama yapmasını sağlar.

- Arama çubuğuna yazıldıkça sonuçlar anlık olarak güncellenir (her tuş vuruşunda filtre uygulanır)
- Arama alanları: kitap başlığı, yazar adı ve etiketler
- Büyük/küçük harf duyarlı arama ("tolkien" → "Tolkien" bulur)
- Firestore verileri cihazda ön belleğe alındığından arama internet bağlantısı gerektirmez

6.2 Filtreleme Seçenekleri

Filtre Türü	Seçenekler
Okuma Durumu	Okuyorum / Okumak İstiyorum / Okudum / Tümü
Etikete Göre	Mevcut etiketler chip olarak listelenir; birden fazla seçilebilir
Puana Göre	★★★★★ ve üzeri / ★★★★ ve üzeri / Puanlanmamış
Eklenme Dönemine Göre	Bu ay / Son 3 ay / Bu yıl / Tüm zamanlar

Aktif filtreler ekranın üstünde chip olarak gösterilir. Her chip'in yanındaki × butonu ile ilgili filtre tek tek kaldırılabilir; "Tümünü Temizle" butonu ile tüm filtreler sıfırlanır.

6.3 Sıralama Seçenekleri

- Eklenme tarihi — Yeniden eskiye (varsayılan)
- Eklenme tarihi — Eskiden yeniye
- Başlık — A'dan Z'ye
- Başlık — Z'den A'ya
- Yazar adı — A'dan Z'ye
- Puan — Yüksekten düşüğe
- Puan — Düşükten yükseğe

6.4 Teknik Not: Firestore Composite Index

Önemli Teknik Detay

Firestore'da birden fazla alana aynı anda filtreleme ve sıralama uygulandığında "Composite Index" oluşturulması gerekir.

Firebase konsolu, bu hatayı otomatik olarak algılar ve logcat'e bir bağlantı gönderir. O bağlantıya tıklandığında index Firebase tarafından otomatik oluşturulur.

Örnek: status == 'read' AND rating >= 4 ORDER BY addedAt DESC
→ Bu kombinasyon için composite index gerekir.

7. Puanlama ve Kişisel Not Sistemi

7.1 Puanlama

- Kitap detay sayfasında 1–5 yıldız arası puanlama bileşeni bulunur
- Puan yalnızca "Okudum" durumundaki kitaplarda etkindir; diğer durumlarda gösterilmez
- Puan istatistik ekranında ve filtreleme sisteminde kullanılır
- Puan değiştirildiğinde Firestore anında güncellenir

7.2 Kişisel Notlar

- Her kitap için serbest metin notu yazılabilir
- Notlar yalnızca kullanıcının kendisine görünür
- Uzunluk sınırı: 1000 karakter (Firestore doküman boyutu optimizasyonu)
- Not varsa kitap listesinde küçük bir "not" ikonu gösterilir

8. İstatistik ve Okuma Hedefi

8.1 İstatistik Ekranı

Kullanıcı profiline bağlı istatistikler, Firestore'daki books koleksiyonu üzerinden hesaplanır:

Metrik	Hesaplama
Toplam kitap	Kütüphanedeki tüm kitapların sayısı
Okunan kitap	status == 'read' olan kayıtlar
Okunmakta olan	status == 'reading' olan kayıtlar
Okuma listesindeki	status == 'to_read' olan kayıtlar
Bu ay okunan	finishedAt >= ayın ilk günü ve status == 'read'
Bu yıl okunan	finishedAt >= yılın ilk günü ve status == 'read'
Ortalama puan	Tüm puanların toplamı / puanlanan kitap sayısı
Toplam sayfa	Okunan kitapların pageCount değerlerinin toplamı

8.2 Yıllık Okuma Hedefi

- Kullanıcı profil ekranında yıllık hedef kitap sayısını belirler (örn: 24 kitap)
- İstatistik ekranında ilerleme çubuğu (ProgressBar) ile görsel takip sağlanır
- "12 / 24 kitap tamamlandı — %50" gibi bir özet gösterilir
- Hedef, yeni yılın başında sıfırlanmaz; kullanıcı manuel olarak güncelleyebilir

9. Uygulama Ekran Akışı

Aşağıda uygulamanın tüm ekranları ve aralarındaki navigasyon akışı özetlenmektedir:

9.1 Kimlik Doğrulama Akışı

- Splash Screen → Firebase oturum kontrolü
- Oturum varsa → Ana Ekran (Bottom Navigation)
- Oturum yoksa → Giriş Yap Ekranı
 - Giriş Yap → Kayıt Ol (yeni kullanıcı)

- Giriş Yap → Şifremi Unuttum → E-posta gönderildi bildirimi
- Kayıt Ol başarılı → Ana Ekran'a yönlendirme

9.2 Ana Ekran Yapısı (Bottom Navigation)

- Kütüphanem — 3'lü sekme (Okuyorum / İstiyorum / Okudum)
- Arama — Google Books API arama ekranı
- İstatistikler — Okuma hedefi ve metrikler
- Profil — Kullanıcı bilgileri, etiket yönetimi, çıkış

9.3 Kitap Detay Akışı

- Kütüphane listesinden kitaba dokunulur → Detay sayfası açılır
- Detay sayfasında: kapak, başlık, yazar, özet, durum seçici, puan, not, etiketler
- Düzenle butonu → Etiket ve not düzenleme modu
- Sil butonu → Onay dialogu → Firestore'dan kalıcı silme

10. 5 Haftalık Geliştirme Planı

Yıldızlı (★) görevler, projeye sonradan eklenen yeni özelliklerdir.

Hafta 1 — Proje Kurulumu + Kimlik Doğrulama [Temel Altyapı]

- Android Studio kurulumu, yeni proje oluşturma (Empty Activity)
- Firebase projesi oluşturma ve google-services.json ekleme
- Firebase Authentication ve Firestore bağımlılıklarını build.gradle'a ekleme
- Kayıt Ol ekranı: ad, e-posta, şifre alanları + Firebase createUserWithEmailAndPassword()
- Giriş Yap ekranı: e-posta, şifre + Firebase signInWithEmailAndPassword()
- Şifremi Unuttum ekranı: sendPasswordResetEmail()
- Oturum kontrolü: Uygulama açılışında kullanıcı durumuna göre yönlendirme
- Basit bir Ana Ekran (içerik yok; yalnızca Bottom Navigation iskeleti)

Cumartesi sunumu: Kayıt / giriş / şifre sıfırlama akışının canlı demosu

Hafta 2 — Kitap Ekleme & Google Books Entegrasyonu [API Entegrasyonu]

- Google Books API key alımı (Google Cloud Console) ve Retrofit kurulumu
- Kitap arama ekranı: arama çubuğu + API çağırısı + RecyclerView sonuç listesi
- Kitap seçildiğinde detay bilgilerin forma otomatik doldurulması
- Firestore'da books alt koleksiyonu oluşturma ve kitap kaydetme
- Kitap detay sayfası: kapak görseli (Glide/Coil), başlık, yazar, özet
- ★ ★ **Kitap eklerken etiket girişi: ChipGroup + EditText kombinasyonu**
- ★ ★ **Etiketlerin Firestore'daki kitap dokümanına tags dizisi olarak kaydedilmesi**

Cumartesi sunumu: Arama yapıp kitap + etiket ekleme akışı demosu

Hafta 3 — 3'lü Liste + Barkod + Kütüphane Arama [Temel İşlevler]

- Ana ekranda TabLayout ile 3 sekme: Okuyorum / İstiyorum / Okudum
- Her sekme için ayrı Fragment ve Firestore sorgusu (status filtresi)
- Kitap durumunu detay sayfasından değiştirme (dropdown menü)

- ML Kit Barcode Scanning kütüphanesi ekleme ve kamera izni isteme
 - Barkod okunduğunda ISBN ile Google Books API sorgusu ve otomatik bilgi doldurma
 - ★ ★ **Kütüphane içi arama çubuğu: başlık, yazar ve etiket üzerinde anlık arama**
 - ★ ★ **Etiket chip'leri ile hızlı filtre: chip'e dokunulduğunda o etikete ait kitaplar listelenir**
- Cumartesi sunumu: 3'lü liste + barkodla kitap ekleme + etiketle filtreleme demosu*

Hafta 4 — Puanlama, Notlar, Filtre & Sıralama [İçerik Zenginleştirme]

- 5 yıldız puanlama bileşeni (RatingBar veya özel chip bileşeni)
 - Kişisel not ekleme / düzenleme alanı (multiline EditText)
 - ★ ★ **Filtre paneli (BottomSheetDialog): durum, etiket, puan aralığı, ekleme tarihi**
 - ★ ★ **Sıralama seçenekleri: eklenme tarihi, alfabetik, puan, yazar**
 - ★ ★ **Aktif filtreleri gösteren chip listesi ve 'Tümünü Temizle' butonu**
 - ★ ★ **Composite index oluşturma (Firestore konsolu üzerinden)**
 - İstatistik ekranı: toplam, okunan, aylık, yıllık sayaçlar
 - Yıllık okuma hedefi: girdi alanı + ProgressBar
- Cumartesi sunumu: Tam filtre + sıralama + istatistik ekranı demosu*

Hafta 5 — Cilalama, Testler & Sunum Hazırlığı [Bitiş]

- UI tutarlılığı gözden geçirme: renkler, fontlar, boşluklar, ikonlar
 - Boş durum ekranları: hiç kitap yok, arama sonucu bulunamadı
 - Loading indicator (CircularProgressIndicator) tüm API çağrılarında
 - Hata ekranları: internet yok, API hatası, Firebase hatası
 - ★ ★ **Etiket yönetim ekranı: tüm etiketleri görme, kullanılmayanları silme**
 - Kitap silme özelliği + onay dialogu
 - Uygulama ikonu tasarımı (Android Studio Image Asset)
 - Splash screen (Lottie animasyonu veya statik logo)
 - Final demo senaryosu hazırlama ve prova
- Cumartesi sunumu: Tüm özellikler çalışır halde — final sunum*

11. Geliştirme Notları ve Öneriler

11.1 Kotlin'e Başlarken

Kotlin bilgisi düşük olan geliştiriciler için önerilen yaklaşım:

- Her ekranı tek bir Activity veya Fragment olarak düşün
- Bir sonraki haftaya geçmeden önce mevcut haftanın tüm görevlerini anla
- Yazılan her kod bloğunun ne iş yaptığını yorumla açıkla (kod okunabilirliği)
- ChatGPT veya Claude ile kod alırken "Bu kod ne yapıyor?" diye sor
- Logcat'i sık kullan — hatalar orada görünür

11.2 Firebase Güvenlik Kuralları

Geliştirme sürecinde Firestore Security Rules şu şekilde ayarlanmalıdır:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId}/{document=**} {
```

```
allow read, write: if request.auth != null
    && request.auth.uid == userId;
}
}
}
```

Bu kural, her kullanıcının yalnızca kendi verilerine erişebilmesini garanti eder.

11.3 Önemli Bağımlılıklar (build.gradle)

```
// Firebase
implementation 'com.google.firebase:firebase-auth-ktx'
implementation 'com.google.firebase:firebase-firestore-ktx'

// Google Books API (Retrofit)
implementation 'com.squareup.retrofit2:retrofit:2.9.0'
implementation 'com.squareup.retrofit2:converter-gson:2.9.0'

// Barkod Okuma
implementation 'com.google.mlkit:barcode-scanning:17.2.0'
implementation 'androidx.camera:camera-camera2:1.3.0'

// Görsel Yükleme
implementation 'io.coil-kt:coil:2.5.0'

// Navigation Component
implementation 'androidx.navigation:navigation-fragment-ktx:2.7.0'
implementation 'androidx.navigation:navigation-ui-ktx:2.7.0'
```

11.4 Sık Yapılan Hatalar ve Çözümleri

google-services.json eksik

Çözüm: Firebase'den indirilen dosyayı app/ klasörüne (modül kökü) koymak gerekir

Firestore izin hatası (PERMISSION_DENIED)

Çözüm: Security Rules'un doğru ayarlandığını ve kullanıcının giriş yapmış olduğunu kontrol et

Google Books API 403 hatası

Çözüm: API key'in Google Cloud Console'da Books API için etkinleştirildiğini kontrol et

Barkod tarama çalışmıyor

Çözüm: AndroidManifest.xml'e kamera izni eklenmesi gerekir: <uses-permission android:name='android.permission.CAMERA'/>

Composite index hatası

Çözüm: Logcat'teki Firebase linkine tıkla, index otomatik oluşturulur (~2 dakika)